

# Sistema Gestione Segnalazioni - Tao Zhang

---

## La motivazione della scelta dell'ADT

Per la realizzazione del progetto è stata scelta una struttura dati dinamica basata su **lista concatenata**. La scelta di questo ADT è motivata dalla necessità di gestire un numero variabile di segnalazioni senza imporre limiti fissi alla memoria utilizzata.

Ogni segnalazione rappresenta un nodo della lista e contiene tutte le informazioni necessarie: **codice identificativo, codice completo, nome utente, categoria, descrizione, data, livello di urgenza e stato della segnalazione**.

La lista concatenata permette inserimenti rapidi in testa, ricerche sequenziali semplici e rimozione dinamica degli elementi senza riallocazioni complesse. Questa soluzione risulta particolarmente adatta ad un sistema di gestione comunale dove il numero di segnalazioni può crescere dinamicamente nel tempo.

## Come usare e interagire con il sistema

### Accesso al sistema

1. Avviare il programma eseguibile con nome **"progettoPSD.exe"**.
2. Alla schermata iniziale scegliere una delle seguenti opzioni:
  - Login
  - Registrazione
3. Selezionando la registrazione, l'utente crea un nuovo account personale.
4. Selezionando il login, l'utente può accedere all'account creato o in uno già esistente.
  - *Utenti già esistenti:*
    - Username: **user, user1**
    - Password: **u**
  - *Amministratori già esistenti:*
    - Username: **admin**
    - Password: **a**
5. Dopo il login il programma riconosce automaticamente:
  - utente normale.
  - amministratore.

## Funzionalità Utente

L'utente può utilizzare le seguenti operazioni:

- Creare una segnalazione:
  - *Inserendo categoria del problema, descrizione e livello di urgenza. Il sistema genera automaticamente un codice identificativo*
- Visualizzare le proprie segnalazioni:
  - *Mostra tutte le segnalazioni aperte dall'utente autenticato*
- Ricercare una segnalazione:
  - *Inserendo il codice numerico della segnalazione*
- Controllare lo stato di una segnalazione:
  - Verificare se la segnalazione è:
    - aperta
    - in lavorazione
    - chiusa

## Funzionalità Amministratore

L'amministratore dispone di tutte le funzionalità utente più le seguenti operazioni:

- Visualizzare tutte le segnalazioni
- Ricercare segnalazioni per categoria
- Aggiornare lo stato di una segnalazione
- Eliminare segnalazioni chiuse
- Generare il report finale del sistema

## Salvataggio dei dati

- Tutte le segnalazioni vengono salvate automaticamente nel file:
  - **segnalazioni.txt**
- Al riavvio del programma le segnalazioni vengono ricaricate automaticamente in memoria.

## Gestione amministratore

Per motivi di sicurezza, durante la registrazione tutti gli utenti vengono creati come utenti normali. L'unico modo per assegnare i privilegi di amministratore è modificare manualmente il valore associato all'utente nel file di gestione account:

- **0 → utente normale**
- **1 → amministratore**

Questa modifica deve essere effettuata direttamente nel file "**account.txt**" modificando l'ultimo parametro presente (es. "admin a 0" → "admin a 1").

## Progettazione del sistema

Il progetto è stato sviluppato in **linguaggio C** seguendo una progettazione modulare. Ogni modulo è stato separato in file **.c** e **.h** per migliorare leggibilità, manutenibilità e organizzazione del codice.

- Il modulo **account** gestisce registrazione, login e l'autenticazione utenti.
- Il modulo **segnalazione** contiene tutte le operazioni CRUD, la gestione delle priorità, il filtraggio, il caricamento e il salvataggio su file.
- Il modulo **codice** genera automaticamente codici identificativi univoci composti da prefisso categoria, data corrente e numero casuale.
- Il modulo **utils** contiene funzioni di utilità come input sicuri, pulizia schermo e gestione dei menu.
- *Il programma utilizza file di testo per la persistenza dei dati, consentendo il automatico recupero delle segnalazioni al riavvio del sistema.*

## Specifica sintattica e semantica

### **creaSegnalazione(char username[])**

Input: username dell'utente autenticato.

Output: nuova struttura Segnalazione allocata dinamicamente.

Pre-condizione: utente autenticato.

Post-condizione: segnalazione salvata nel file.

### **aggiungiSegnalazione(Segnalazione\* head, char username[])**

Input: testa lista e username.

Output: nuova testa della lista.

Effetto: inserimento della segnalazione nella lista.

### **stampaSegnalazioni(Segnalazione\* head, char username[], int isAdmin)**

Input: lista segnalazioni, username e permessi admin.

Output: visualizzazione delle segnalazioni.

Effetto: stampa filtrata in base ai permessi.

### **stampaSegnalazione(Segnalazione\* s)**

Input: puntatore ad una segnalazione.

Output: stampa dettagli della segnalazione.

### **statoSegnalazioneUtente(Segnalazione\* head, char username[])**

Input: lista segnalazioni e username.

Output: stato della segnalazione selezionata.

Pre-condizione: la segnalazione deve appartenere all'utente.

**cercaPerCodice(Segnalazione\* head, int codice)**

Input: lista e codice numerico.

Output: puntatore alla segnalazione trovata oppure NULL.

**cercaPerCategoria(Segnalazione\* head)**

Input: lista segnalazioni.

Output: stampa delle segnalazioni appartenenti alla categoria selezionata.

**aggiornaStato(Segnalazione\* head, int isAdmin)**

Input: lista segnalazioni e privilegi admin.

Output: aggiornamento stato.

Pre-condizione: utente amministratore.

**stampaAperteAdmin(Segnalazione\* head)**

Input: lista segnalazioni.

Output: stampa delle segnalazioni aperte.

**stampaPerStato(Segnalazione\* head, char stato[])**

Input: lista e stato richiesto.

Output: stampa delle segnalazioni con lo stato selezionato.

**stampaUrgenti(Segnalazione\* head)**

Input: lista segnalazioni.

Output: stampa delle segnalazioni urgenti e meno urgenti.

**eliminaSegnalazione(Segnalazione\* head, int isAdmin)**

Input: lista segnalazioni e privilegi admin.

Output: nuova lista aggiornata.

Pre-condizione: solo segnalazioni chiuse eliminabili.

**generaReport(Segnalazione\* head)**

Input: lista segnalazioni.

Output: statistiche generali del sistema.

**salvaSegnalazione(Segnalazione\* s)**

Input: segnalazione.

Output: salvataggio nel file di testo.

**caricaSegnalazioni()**

Input: file segnalazioni.txt.

Output: lista concatenata caricata in memoria.

## Razionale dei casi di test

I casi di test sono stati progettati per verificare il corretto funzionamento di tutte le funzionalità implementate.

Sono stati effettuati test per: Registrazione corretta di una segnalazione Ricerca di segnalazioni esistenti e inesistenti Aggiornamento corretto dello stato Gestione delle priorità urgenti Visualizzazione filtrata per utenti e amministratori Generazione del report finale

Sono stati inoltre verificati casi di errore, come codici inesistenti, accessi senza privilegi amministrativi e file mancanti.

## Sistema operativo e Makefile

Il progetto ed il Makefile sono stati sviluppati e testati su ambiente **Windows** utilizzando **MSYS2 UCRT64** e **compilatore GCC**.

La compilazione avviene tramite Makefile con il comando:

**make**

*Il sistema utilizza compilazione modulare dei file sorgente e linking automatico.*